NON BOUSSINESQ PLUMES AND FOUNTAINS

Dans le cadre de l'enseignement de MFEI

Catusse Lucas - Sanchez Alexandre

February 25th 2026

Polytech Lyon – Mechanics, 5th year
Supervised by M.Landel

Contribution Statement

We have both contributed equally to the preparation of this report.

Alexandre was primarily responsible for the volcano case (Section 5.5) and the Non-Boussinesq cases (Sections 4.2, 4.3, and 4.4). Lucas was primarily responsible for the fountain cases (Section 5.3), the Deepwater Horizon case (Section 5.4), and the Boussinesq case (Section 4.1).

All remaining parts of the report, including model development, analysis, and manuscript preparation, were completed collaboratively.

Agreed mark distribution: 2/4 for Alexandre & 2/4 for Lucas

Date: _____

Signature (Alexandre): _____

Signature (Lucas): _____

Abstract

This mini-project studies the dynamics of non-Boussinesq plumes and fountains. Its purpose is to understand the impact of large density differences between a turbulent buoyant flow and the ambient fluid. Classical plume theory often relies on the Boussinesq approximation (assuming small density variations except in the buoyancy term : $\rho_a - \rho_p << \rho_a$). However, for many geophysical and industrial flows such as volcanic eruptions, this approximation is no longer valid. Thus, studying non-Boussinesq effects is essential to improve the physical description of such flows and predict their behavior .

The problem is formulated using the conservation equations of volume, mass, and momentum for an axisymmetric turbulent plume. We first analyze the stationary Boussinesq case with a Top-hat profile (w and ρ uniform over a width b ; 0 elsewhere). The equations are then solved numerically (on Python with explicit Euler and Runge-Kutta 4 method) in order to simulate both Boussinesq and non-Boussinesq configurations. Different boundary conditions are considered through various numerical scenarios. Firstly, theoretical cases are studied to validate our Python scripts. Secondly, more advanced real case scenarios are solved, allowing us to observe complex phenomena.

The results show that relaxing the Boussinesq approximation modifies plume half-width, speed, and buoyancy flux variations, particularly when density contrasts are large. In the non-Boussinesq regime, entrainment and momentum transfer are strongly affected by the density ratio. This leads to differences in plume structure. In the case of fountains, we observed that the maximum rise height depends on source momentum and density contrast.

Overall, this study highlights the limits of the Boussinesq approximation. It demonstrates the importance of accounting for density variations in strongly buoyant flows. This field of study can be used for both environmental and industrial applications.

Contents

1 Introduction

Plumes and turbulent fountains have a fundamental role in environmental and industrial fluid mechanics. They are present in many natural situations, such as volcanic eruptions, fires, industrial exhausts, natural ventilation of buildings, and deep-sea releases. These flows are characterized by the entrainment of ambient fluid, which modifies their structure, velocity, width, and vertical evolution.

Most classical theoretical studies of plumes rely on the Boussinesq approximation, which assumes that density variations are small ($\rho_a - \rho_p \ll \rho_a$) and only influence the buoyancy term. While this assumption is valid in many situations, it becomes inaccurate when density contrasts are large, such as in volcanic plumes, water fountains, or very hot gas jets. In these cases, a non-Boussinesq formulation is required to correctly describe inertia effects, entrainment processes, and momentum transfer.

The main objective of this work is to study and compare the dynamics of turbulent plumes and fountains under both Boussinesq and non-Boussinesq assumptions, using the conservation equations. The project aims to quantify the impact of modelling assumptions on the evolution of the characteristic fluxes, plume structure, and maximum rise height, considering both homogeneous and stratified ambient environments.

First, the Boussinesq case is studied. The governing equations are reformulated in terms of mass flux, momentum flux, and buoyancy flux, and solved numerically. The influence of ambient stratification is investigated. For this matter, we study the case of buoyant plumes and negatively buoyant jets (fountains). These models are then applied to physically relevant configurations.

In a second stage, the non-Boussinesq approach is developed in order to analyse flows with strong density contrasts. The entrainment coefficient is modified to account for density effects and the local density is used both for inertia and buoyant terms. This formulation is applied to realistic cases such as volcanic plumes and gas leak, allowing the analysis of specific phenomena such as necking.

This work builds upon the theoretical foundations established by Morton, Taylor and Turner (1956) [1]. It also uses developments on fountains and non-Boussinesq plumes from various authors, including Bloomfield and Kerr [7], List [8], Scase et al. [9], van den Bremer and Hunt [10], Woods [11, 3], Baines and Turner [12], and Rooney and Linden [13].

The report is organised as follows. The first part presents the general theory of turbulent plumes and the Boussinesq approximation. The second part develops the non-Boussinesq formulation. The third part presents the numerical methods used for implementation. The fourth part presents the various scenarios studied. Finally, a general discussion compares the different regimes and highlights the limitations and perspectives of the models studied.

2 Method

2.1 Boussinesq Case

First, we study the case of Boussinesq plumes. A plume is said to be Boussinesq if and only if density variations are negligible compared to the reference density ($\Delta\rho = \rho_a - \rho_p \ll \rho_a$).

Boussinesq Approximation

Here we consider $\Delta\rho = \rho_a - \rho_p \ll \rho_a \implies \rho \approx \text{const.}$

The Boussinesq approximation is a simplified model used in buoyancy-driven flows. It assumes that density variations are small enough to be neglected in the continuity and momentum equations, except when multiplied by gravity in the buoyancy term. This approximation simplifies the governing equations while remaining physically accurate for many geophysical flows.

The flow dynamics are governed by the reduced gravity defined as $g' = g \times \frac{\rho_a - \rho_p}{\rho_a}$.

We redefine here the flux equations Q, M, and F as follows:

$$Q_v = \pi b^2 \langle \omega \rangle \tag{1}$$

$$M_v = \pi b^2 \langle \omega \rangle^2 \tag{2}$$

$$F = Q \times g' \quad \text{with} \quad g' = g \times \frac{\rho_a - \rho_p}{\rho_a} \tag{3}$$

2.1.1 Asymmetric Jet Model: "Top-Hat" Profile or Gaussian Profile?

Top Hat Profile

The Top-Hat profile assumes that the vertical velocity and ρ are constant across the plume's cross-section and drop to zero abruptly at the boundary ($r = b$). This simplification allows us to transform partial differential equations into a system of ordinary differential equations (ODEs).

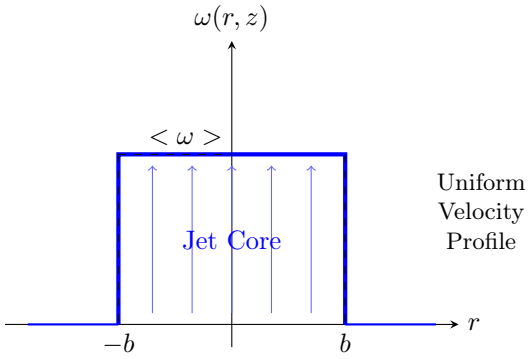

Figure 1: Top Hat Profile : The velocity and density are uniform across the entire jet cross-section.

- Velocity :

$$\omega(r,z) = \begin{cases} \langle \omega \rangle(z) & \text{if } r < b, \\ 0 & \text{else} \end{cases}$$

- Density :

$$\rho(r,z) = \begin{cases} \rho_p(z) & \text{if } r < b, \\ \rho_a & \text{else} \end{cases} \qquad \text{with } \rho_p \approx \rho_a$$

- Entrainment : the lateral velocity satisfies

$$u_e = \alpha\langle\omega\rangle$$

The Morton-Taylor-Turner (MTT) [1] entrainment hypothesis states that the inflow velocity of the surrounding fluid (u_e) is directly proportional to the characteristic vertical velocity of the plume through the entrainment coefficient α.

Gaussian Profile

Why is the Gaussian profile more consistent?

In the *Top-Hat* model, the velocity w and the buoyancy are assumed to be constant across the entire plume width b, and to drop abruptly to zero outside this region.

In reality, turbulence does not stop abruptly. Turbulent mixing naturally smooths radial gradients.

- **Turbulent diffusion:** The plume center moves faster, while the edges are slowed down by mixing with the surrounding fluid. This naturally leads to a Gaussian profile.

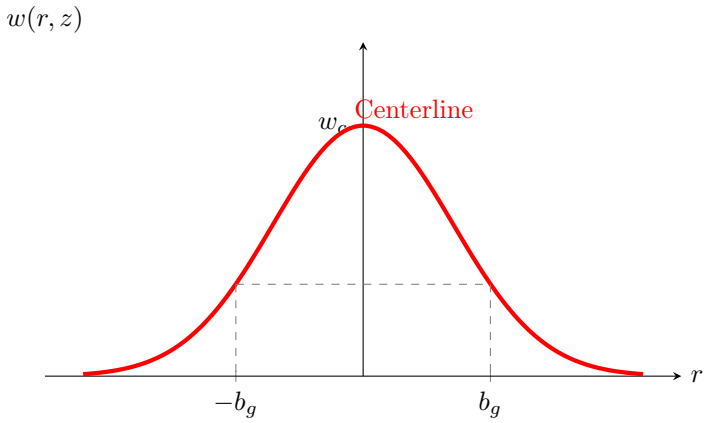

Figure 2: Gaussian velocity profile

The velocity decreases smoothly, representing momentum diffusion into the ambient fluid.

Integral Formulation of the Fluxes

Assuming Gaussian radial profiles for velocity and buoyancy, the fluxes integrated over the jet cross-section are written as

$$Q = \pi b^2 w_m,$$

$$M = \frac{1}{2}\pi b^2 w_m^2,$$

$$F = \frac{1}{2}\pi b^2 w_m g'_m.$$

Unlike the *Top-Hat* model, the integration of Gaussian profiles introduces shape coefficients, reflecting the gradual radial decay of the dynamical quantities within the plume.

Governing Equation System

Using these integral definitions, the jet evolution system retains its general structure:

$$\frac{dQ}{dz} = 2\alpha\sqrt{2}\,\pi b w_m,$$

$$\frac{dM}{dz} = F,$$

$$\frac{dF}{dz} = 0,$$

the differences with the uniform model appearing only as multiplicative constants related to the chosen radial profile.

Profile Choice for the Remainder of the Study

Although the Gaussian profile provides a more realistic description of the internal structure of a turbulent jet, it introduces additional shape coefficients without modifying the overall physics of the system.

In order to retain a simple analytical formulation, the Top-Hat profile will therefore be adopted for all analyses presented in the remainder of this work.

We now derive the evolution equations for Q, M, and F with respect to z in order to obtain the flux equations.

2.1.2 Conservation of Volume Flux

The continuity equation in cylindrical (axisymmetric) coordinates is written as

$$\frac{1}{r}\frac{\partial(ru)}{\partial r} + \frac{\partial \omega}{\partial z} = 0.$$

Multiplying by $2\pi r$ and integrating between 0 and $+\infty$ gives

$$2\pi \int_0^{+\infty} \left(\frac{\partial(ru)}{\partial r} + r\frac{\partial \omega}{\partial z} \right) dr = 0.$$

The first term becomes $2\pi R u(R)$. Using the entrainment condition at the jet boundary ($u(R) = -u_e$ and $R = b$), we obtain

$$2\pi R u(R) = 2\pi b(-u_e) = -2\pi b u_e.$$

The second term becomes

$$\frac{d}{dz}\left(\int_0^b 2\pi r \langle \omega \rangle \, dr \right).$$

By definition of the volume flux, $Q_v(z) = \int_0^b 2\pi r \langle \omega \rangle \, dr$. We therefore obtain

$$\frac{d}{dz}\int_0^b 2\pi r \langle \omega \rangle \, dr = \frac{dQ_v}{dz}.$$

Using these results together with the expression of M, we finally obtain

$$\frac{dQ_v}{dz} = 2\alpha\sqrt{\pi M}.$$

By analogy, the governing system of equations becomes

$$\boxed{\begin{aligned}\frac{dQ_v}{dz} &= 2\alpha\sqrt{\pi M}\\ \frac{dM_v}{dz} &= \frac{FQ}{M}\\ \frac{dF}{dz} &= 0\end{aligned}}$$

2.1.3 Solution Using the Power-Law Method

To simplify the analysis, we assume the following initial conditions: $Q(0) = 0$ and $M(0) = 0$.

We seek self-similar solutions in which the plume properties evolve as power laws of the vertical coordinate z. This behavior is characteristic of the far-field region of a turbulent plume.

Let us assume

$$Q(z) = Az^n \quad \text{and} \quad M(z) = Bz^m.$$

Substituting these expressions for $Q(z)$ and $M(z)$ into the previously obtained system of equations requires equality of powers of z, meaning that the exponents must be identical on both sides of each equation. This leads to the following system for the exponents:

1. $nAz^{n-1} = 2\pi\alpha\sqrt{B}z^{m/2} \;\Rightarrow\; n - 1 = \frac{m}{2}$
2. $mBz^{m-1} = F_0\frac{A}{B}z^{n-m} \;\Rightarrow\; m - 1 = n - m$

Solving this system yields $m = \frac{4}{3}$ and $n = \frac{5}{3}$.

The constants A and B now incorporate the factor π, leading to

$$M(z) = \left(\frac{9\pi\alpha}{10}\right)^{2/3} F_0^{2/3} z^{4/3},$$

$$Q(z) = \frac{6\pi\alpha}{5}\left(\frac{9\pi\alpha}{10}\right)^{1/3} F_0^{1/3} z^{5/3}.$$

Since, in both expressions, the only independent variable is z, we obtain the scaling laws $M(z) \propto z^{4/3}$ and $Q(z) \propto z^{5/3}$.

2.1.4 Physical Quantities

The jet radius (without the factor π in the definition of b) is given by

$$b(z) = \sqrt{\frac{Q^2}{M}} = \frac{6\pi\alpha}{5}z.$$

The radius increases linearly with height, confirming the conical structure of the plume. The spreading rate is governed by the entrainment coefficient α.

The mean velocity is

$$\langle\omega\rangle = \frac{M}{Q} = \frac{5}{6\pi\alpha}\left(\frac{10}{9\pi\alpha}\right)^{1/3} F_0^{1/3} z^{-1/3}.$$

The velocity decreases as $z^{-1/3}$, showing that although the plume gains mass through entrainment, it decelerates due to the redistribution of momentum over an increasingly large cross-section.

The reduced gravity is given by $g' = \frac{F_0}{Q} \propto z^{-5/3}$.

2.1.5 Stratified Environment

Previously, the ambient density was assumed to be constant ($\rho_a = \text{const}$). In a homogeneous environment, the buoyancy flux F is conserved ($dF/dz = 0$). However, in a stratified fluid such as the ocean, the ambient density $\rho_a(z)$ varies with depth, directly affecting the driving force of the plume.

We now consider a stratified environment such that

$$\rho_a(z) = \rho_a(0) - \gamma z,$$

where γ represents the vertical density gradient. The stratification strength is characterized by the Brunt–Väisälä frequency, defined as

$$N^2 = -\frac{g}{\rho_a(0)}\frac{d\rho_a}{dz}.$$

This stratification does not affect the first two governing equations (for Q and M). However, it modifies the evolution equation of the buoyancy flux. First, conservation of mass gives

$$\frac{d}{dz}(\rho_p Q) = \rho_a(z)\,\frac{dQ}{dz}.$$

Starting from the definition of the buoyancy flux associated with the reduced gravity g',

$$F = Qg' = Qg\frac{\rho_a(z) - \rho_p}{\rho_a(0)},$$

we obtain $F = \frac{g}{\rho_a(0)}\left[Q\rho_a(z) - Q\rho_p\right]$. Differentiating with respect to z yields

$$\frac{dF}{dz} = \frac{g}{\rho_a(0)}\left(\frac{d(Q\rho_a)}{dz} - \frac{d(Q\rho_p)}{dz}\right).$$

Applying the product rule to the first term and using mass conservation for the second term gives

$$\frac{d(Q\rho_p)}{dz} = \rho_a(z)\frac{dQ}{dz}.$$

Substituting this relation leads to

$$\frac{dF}{dz} = \frac{g}{\rho_a(0)}\left(Q\frac{d\rho_a}{dz} + \rho_a\frac{dQ}{dz} - \rho_a\frac{dQ}{dz}\right).$$

The terms involving $\rho_a\frac{dQ}{dz}$ cancel, yielding the fundamental relation

$$\boxed{\frac{dF}{dz} = Q\left(\frac{g}{\rho_a(0)}\frac{d\rho_a}{dz}\right)}$$

This equation shows that variations in buoyancy flux depend solely on the ambient density gradient. Introducing the Brunt–Väisälä frequency, we obtain the form used in the numerical simulations:

$$\frac{dF}{dz} = -QN^2.$$

The negative sign indicates a loss of buoyancy as the plume rises. As the plume entrains surrounding fluid while moving into lighter ambient layers, the density difference $\Delta\rho$ progressively decreases, eventually leading to plume trapping at the level of neutral buoyancy. The term QN^2 therefore represents the buoyancy loss induced by entrainment. Finally, the governing flux system becomes:

$$\boxed{\begin{aligned}\frac{dQ}{dz} &= 2\alpha\sqrt{\pi M}\\ \frac{dM}{dz} &= \frac{FQ}{M}\\ \frac{dF}{dz} &= -QN^2\end{aligned}}$$

2.1.6 Two-Dimensional Line Source

Similarly, for a line source under the Boussinesq approximation ($\rho \approx \rho_a$), the fluxes are defined per unit length. The geometric definitions become:

$$Q = \rho_a b\langle w\rangle, \quad M = \rho_a b\langle w\rangle^2, \quad F = g(\rho_a - \rho)b\langle w\rangle$$

Substituting the standard Morton-Taylor-Turner entrainment velocity ($u_e = \alpha\langle w\rangle$) into the conservation laws gives the following equations:

- **Mass Conservation:**

  The mass entrained per unit height and length is $2\rho_a u_e$. Substituting the standard entrainment velocity:

  $$\frac{dQ}{dz} = 2\rho_a\left(\alpha\langle w\rangle\right) = 2\alpha\rho_a\langle w\rangle$$

  Then, when we replace $\langle w\rangle$ by its expression ($\langle w\rangle = M/Q$), we get:

  $$\frac{dQ}{dz} = 2\alpha\rho_a\frac{M}{Q}$$

- **Momentum and Buoyancy:**

  Momentum conservation and buoyancy flux equations do not change from the point source. Thus, $\frac{dM}{dz} = \frac{FQ}{M}$ and $\frac{dF}{dz} = 0$ (assuming an unstratified ambient fluid).

Finally we obtain this system of equations for the Boussinesq line source:

$$\boxed{\begin{aligned}\frac{dQ}{dz} &= 2\alpha\rho_a\frac{M}{Q}\\ \frac{dM}{dz} &= \frac{FQ}{M}\\ \frac{dF}{dz} &= 0\end{aligned}}$$

2.2 Non-Boussinesq Cases

We then considered Non-Boussinesq cases. Whereas the Boussinesq approximation supposed $\Delta\rho \ll \rho_a$, the Non-Boussinesq approach is really important when the difference between ambient and plume density is large. For example, a fountain of water into air could not be treated as Boussinesq since water is much denser than air.

In such situations, it is necessary to include the density explicitly in the model and hence work in terms of the mass, momentum, and buoyancy fluxes.

Furthermore, the entrainment assumption must be modified. With large density contrasts, the entrainment depends on the density ratio. Thus, the entrainment velocity u_e of the ambient fluid is equal to the square root of the density ratio between the plume density ρ and the ambient density ρ_a:

$$u_e = \alpha\langle w\rangle\sqrt{\frac{\rho}{\rho_a}}$$

We applied this modified entrainment to two different geometries: the axisymmetric point source and the two-dimensional line source.

2.2.1 Axisymmetric Point Source

When we substitute this modified entrainment speed into our mass and momentum conservation equations, we get a new system of ordinary differential equations. First, let us recall the definitions of the mass (Q), momentum (M), and buoyancy (F) fluxes for an axisymmetric point source:

$$Q = \rho b^2 \langle w \rangle, \quad M = \rho b^2 \langle w \rangle^2, \quad F = g(\rho_a - \rho) b^2 \langle w \rangle$$

We can then substitute in the governing equations for a top hat profile.

- **Mass Conservation:**

  When we replace $\rho b^2 \langle w \rangle$ by Q and u_e by $\alpha \langle w \rangle \sqrt{\frac{\rho}{\rho_a}}$, we get:

  $$\frac{dQ}{dz} = 2\rho_a b \left(\alpha \langle w \rangle \sqrt{\frac{\rho}{\rho_a}} \right) = 2\alpha \langle w \rangle b \sqrt{\rho \rho_a}$$

  Moreover, we know from the momentum flux definition that $\sqrt{M} = \langle w \rangle b \sqrt{\rho}$. Finally we get:

  $$\frac{dQ}{dz} = 2\alpha \sqrt{M \rho_a}$$

- **Momentum Conservation:**

  When we replace $\rho b^2 \langle w \rangle^2$ by M in the momentum conservation equation, we obtain:

  $$\frac{dM}{dz} = g b^2 (\rho_a - \rho) = \frac{F}{w}$$

  Then, we know the vertical velocity is the ratio of the momentum and mass fluxes ($w = \frac{M}{Q}$). So finally, the momentum equation becomes:

  $$\frac{dM}{dz} = \frac{FQ}{M}$$

- **Buoyancy Conservation:**

  For Non-Boussinesq cases, we considered an unstratified environment where the ambient density ρ_a is constant. Therefore, the buoyancy flux remains conserved throughout the ascent:

  $$\frac{dF}{dz} = 0$$

Finally we obtain this system of equations for the point source:

$$\boxed{\begin{aligned} \frac{dQ}{dz} &= 2\alpha \sqrt{M \rho_a} \\ \frac{dM}{dz} &= \frac{FQ}{M} \\ \frac{dF}{dz} &= 0 \end{aligned}}$$

We can observe that we obtain the exact same equation as for the Boussinesq approximation cases. Only b and ρ calculation will change.

2.2.2 Two-Dimensional Line Source

Similarly, for a line source, the fluxes are defined per unit length. The geometric definitions become:

$$Q = \rho b \langle w \rangle, \quad M = \rho b \langle w \rangle^2, \quad F = g(\rho_a - \rho) b \langle w \rangle$$

Substituting the modified entrainment velocity u_e into the conservation laws gives a slightly different mass equation, while momentum and buoyancy forms remain similar.

- **Mass Conservation:**

  The mass entrained per unit height and length is $2\rho_a u_e$. Substituting the entrainment velocity:

$$\frac{dQ}{dz} = 2\rho_a \left(\alpha\langle w\rangle\sqrt{\frac{\rho}{\rho_a}}\right) = 2\alpha\rho_a\langle w\rangle\sqrt{\frac{\rho}{\rho_a}}$$

  Then, when we replace $\frac{\rho}{\rho_a}$ and $\langle w\rangle$ by their expression, we get:

$$\frac{dQ}{dz} = 2\alpha\rho_a\frac{M}{Q}\sqrt{\frac{1}{1+\frac{F}{gQ}}}$$

- **Momentum and Buoyancy:**

  Momentum conservation and buoyancy flux equations do not change from the point source. Thus, $\frac{dM}{dz} = \frac{FQ}{M}$ and $\frac{dF}{dz} = 0$.

Finally we obtain this system of equations for the line source:

$$\boxed{\begin{aligned}\frac{dQ}{dz} &= 2\alpha\rho_a\frac{M}{Q}\sqrt{\frac{1}{1+\frac{F}{gQ}}}\\ \frac{dM}{dz} &= \frac{FQ}{M}\\ \frac{dF}{dz} &= 0\end{aligned}}$$

2.2.3 Thermodynamic Non-Boussinesq Model (Volcanic Eruptions)

Finally, we wanted to simulate volcanic eruptions. However, while the previous non-Boussinesq models assumed a conserved buoyancy flux, explosive volcanic eruptions require a much more complex treatment. As described by Woods [3], these eruptions involve multiphase mixtures of hot solid ash and magmatic gas from the volcano. When the plume rises and entrains cold ambient air, heat is transferred from hot ashes to ambient air. Therefore, it causes fast thermal expansion of the entering air. This phenomenon affects the density of the plume, so a heavy jet can turn into a buoyant plume if enough air is entrained.

To capture this thermodynamic phenomenon, we need to adapt the equations we were using in "basic" Non-Boussinesq cases. The conservation equations must track the enthalpy flux (H) and the solid particle mass fraction on top of the momentum and mass fluxes. Moreover, since the buoyancy flux is not conserved anymore, we need to use the momentum equation primitive form.

According to the literature [3], the governing equations for the macroscopic fluxes become:

- **Mass Conservation:** The entrainment of ambient air remains governed by the same equation since there is a strong contrast between plume and ambient density:

$$\frac{dQ}{dz} = 2\alpha\sqrt{M\rho_a}$$

- **Momentum Conservation:** Once again, the momentum conservation equation remains the same as before. This time we do not simplify it since F is no longer a constant.

$$\frac{dM}{dz} = g(\rho_a - \rho)b^2$$

- **Solid Particle Conservation:** Because the plume only entrains pure ambient air, the mass flux of erupted solid ash remains conserved throughout the entire rise.

$$\frac{d}{dz}[Q(1-n)] = 0$$

  where n is the local gas mass fraction inside the plume.

- **Enthalpy Conservation:** The total heat content of the plume changes only due to the enthalpy brought in by the entrained ambient air at temperature T_a with specific heat $C_{p,a}$.

$$\frac{dH}{dz} = \frac{dQ}{dz} C_{p,a} T_a$$

Finally, we obtain this fully coupled thermodynamic system of equations for the volcanic point source:

$$\boxed{\begin{aligned} \frac{dQ}{dz} &= 2\alpha\sqrt{M\rho_a} \\ \frac{dM}{dz} &= g(\rho_a - \rho)b^2 \\ \frac{d}{dz}[Q(1-n)] &= 0 \\ \frac{dH}{dz} &= \frac{dQ}{dz} C_{p,a} T_a \end{aligned}}$$

Unlike simpler models, the local plume density $\rho(z)$ and radius $b(z)$ must be continuously updated at each spatial step using the ideal gas law and the specific heat capacity of the mixture. This ties the flow dynamics directly to its thermal evolution.

2.3 Numerical Implementation

All the theoretical models and real-world scenarios presented in this study were solved numerically using scripts developed in Python on the Spyder software. To integrate the systems of non-linear ordinary differential equations, we discretized the spatial domain along the vertical axis z into N points separated by a constant step size dz.

In this project, we implemented two distinct numerical integration schemes to compute the spatial evolution of the fluxes:

- **Explicit Euler Method (First-Order):** Given the initial conditions at the source $z = 0$, the fluxes at the next spatial step $n + 1$ are approximated using their derivatives evaluated at the current step n. For example:

$$Q_{n+1} = Q_n + dz \left(\frac{dQ}{dz}\right)_n$$

This first-order approach was utilized to iteratively solve several coupled systems, including the theoretical plumes and the thermodynamic volcanic models.

- **Runge-Kutta Method (Fourth-Order - RK4):** The RK4 algorithm was also implemented for specific scenarios, notably the theoretical fountains and the Deepwater Horizon blowout. This method evaluates four intermediate slopes at each step dz to refine the prediction of the next value:

$$\begin{aligned} k_1 &= f(z_n, y_n) \\ k_2 &= f\left(z_n + \frac{dz}{2}, y_n + \frac{dz}{2}k_1\right) \\ k_3 &= f\left(z_n + \frac{dz}{2}, y_n + \frac{dz}{2}k_2\right) \\ k_4 &= f(z_n + dz, y_n + dz k_3) \end{aligned}$$

$$y_{n+1} = y_n + \frac{dz}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Physical Variables Extraction and Stopping Criteria

Regardless of the integration scheme used, the numerical loops calculate the fluxes (Q, M, and F or H). However, to interpret the flow physics, the local vertical velocity w, mixture density ρ, and effective radius b must be extracted at each height z.

The local vertical velocity is always obtained from the ratio of momentum and mass fluxes:

$$w = \frac{M}{Q}$$

The density extraction depends on the chosen approximation:

- **Boussinesq:** The density is assumed constant for inertia calculations ($\rho \approx \rho_a$).
- **Standard Non-Boussinesq:** Extracted from the buoyancy flux: $\rho = \frac{\rho_a}{1+\frac{F}{gQ}}$.
- **Thermodynamic (Volcanic):** Evaluated using the ideal gas law and the specific heat capacity derived from the enthalpy flux H.

Once ρ and w are known, the half width is calculated geometrically (for example, $b = \sqrt{\frac{Q}{\rho w}}$ for an axisymmetric point source).

Finally, a numerical safeguard is implemented within the loops for dense flows (fountains and collapsing volcanoes). If the upward velocity drops to zero ($w \leq 0$) or the momentum becomes negative, the loop automatically breaks. This allow us to identify the maximum height reached ($z_{\max}$) before the fluid collapses under its own weight.

2.4 Simulation Scenarios and Parameters Selection

To validate our numerical solutions and demonstrate their physical applicability, we designed seven simulation scenarios. These scenarios are divided into two categories. First, theoretical cases intended to give a first try to our equations and programs. Secondly, real-world geophysical applications based on historical data.

For all simulations, the gravitational acceleration is set to $g = 9.81\ \mathrm{m/s^2}$, and the standard entrainment coefficient is $\alpha = 0.085$ [1].

2.4.1 Theoretical Benchmark Cases

These initial cases use empiric values to isolate specific physical behaviors, such as asymptotic growth or buoyancy-driven collapse.

- **Case 1: Simple Plume (Boussinesq)** This case models a slightly warm air plume (lighter than the air) ($\rho_0 = 1.15\ \mathrm{kg/m^3}$) released into air ($\rho_a = 1.204\ \mathrm{kg/m^3}$). Because the density difference is small ($\Delta\rho \ll \rho_a$), the Boussinesq approximation is valid.
- **Case 2: Light Plume (Non-Boussinesq)** To test non-Boussinesq momentum conservation, we simulate the release of pure Helium ($\rho_0 = 0.18\ \mathrm{kg/m^3}$) into air. The important initial buoyancy shows the difference between using the ambient density ρ_a (Boussinesq) and the true local density ρ (Non-Boussinesq) for inertia calculations.
- **Case 4: A Heavy Gas Fountain (SF6) in Air:** A fountain of sulfur hexafluoride (SF6) with density $\rho_{SF6} = 6.12\,\mathrm{kg/m^3}$ injected into ambient air ($\rho_{air} = 1.225\,\mathrm{kg/m^3}$). With a density ratio $\rho_0/\rho_a \approx 5$, this case falls strictly within the Non-Boussinesq regime. The differential system is initialized at the nozzle ($z = 0$) with a radius $b_0 = 0.05\,\mathrm{m}$ and an ejection velocity $w_0 = 5.0\,\mathrm{m/s}$.
- **Case 5: A Saline Water Fountain in Freshwater:** A fountain of saline water ($\rho_s = 1030\,\mathrm{kg/m^3}$) evolving in freshwater ($\rho_d = 1000\,\mathrm{kg/m^3}$). Although this case satisfies the Boussinesq approximation criteria ($\Delta\rho/\rho \approx 3\%$), we have chosen to solve the system using both Boussinesq and Non-Boussinesq models to compare convergence and accuracy of the results. The differential system is initialized at the nozzle ($z = 0$) with a radius $b_0 = 0.05\,\mathrm{m}$ and an ejection velocity $w_0 = 5.0\,\mathrm{m/s}$.

2.4.2 Real-World Applications

Theoretical point-source boundary conditions ($b_0 = 0$) cannot be applied to real geological vents without generating mathematical singularities. Therefore, the macroscopic source fluxes (Q_0, M_0, H_0) for the following scenarios were deduced from field measurements and established literature.

- **Case 5: Deepwater Horizon Blowout (Gulf of Mexico, 2010).** This scenario models a highly buoyant submarine oil spill. For this simulation, we base our study on data from the *Macondo* well, located at a depth of 1500 m. The chosen parameters, taken from NOAA reports and the work of Socolofsky et al. (2011) [4], are as follows: The ejected fluid is a light sweet crude oil with a density of approximately $\rho_{oil} \approx 850\,\mathrm{kg/m^3}$. The source corresponds to a pipeline rupture with a diameter of $D \approx 0.50\,\mathrm{m}$ and an estimated ejection velocity of $w_0 = 0.8\,\mathrm{m/s}$, accounting for the thrust effect of the associated natural gas [5]. The ambient environment is an ocean stratification characterized by a Brunt–Väisälä frequency of $N^2 = 5 \cdot 10^{-5}\,\mathrm{s^{-2}}$.

- **Case 6 : Eyjafjallajökull Volcanic Eruption (Iceland, 2010).** This scenario uses the coupled thermodynamic model. Gudmundsson et al. [6] measured a Mass Eruption Rate (MER) of approximately 7.5×10^5 kg/s during the explosive phase. According to Woods [3], typical magmatic mixtures erupt with a gas mass fraction $n_0 \approx 0.04$ at a temperature $T_0 \approx 1200$ K [2].

  To satisfy the measured MER, the effective volcano radius is dynamically computed by the algorithm ($b_0 \approx 36.5$ m). We defined two sub-cases based on typical exit velocities:

  1. **Case 6.1 (Successful Plume):** An initial velocity of $w_0 = 125$ m/s, providing sufficient momentum to entrain ambient air, reverse the buoyancy through rapid thermal expansion, and rise.
  2. **Case 6.2 (Column Collapse):** A lower velocity of $w_0 = 40$ m/s. This lower momentum reduces the entrainment rate in the lower dense jet region, failing to incorporate and heat enough air before the initial momentum is dissipated, which leads to a dense fountain collapse [3].

2.4.3 Parameters Summary

The initial boundary conditions and calculated physical parameters for all seven simulation scenarios are summarized in Table 1.

Simulation Case	ρ_a **(kg/m^3)**	ρ_0 **(kg/m^3)**	b_0 **(m)**	w_0 **(m/s)**	**Source & Justification**
Part 1: Theoretical Benchmark Cases (Model Validation)					
1. Simple Plume (Boussinesq)	1.204	1.150	0.50	2.0	Warm air plume ($\Delta\rho \ll \rho_a$). Boussinesq validity.
2. Light Plume (Non-Bouss.)	1.204	0.180	0.50	2.0	Pure Helium.
3. Heavy Gas Fountain (SF6)	1.225	6.120	0.05	5.0	High density ratio (≈ 5). Non-Boussinesq regime.
4. Saline Fountain (Benchmark)	1000.0	1030.0	0.05	5.0	Comparison of Bouss. vs Non-Bouss. convergence.
Part 2: Real-World Geophysical Applications					
5. Deepwater Horizon Blowout	1025.0	≈ 850.0	0.25	0.8	Macondo well (1500m). $N^2 = 5 \cdot 10^{-5}$ s^{-2}. [4], NOAA reports.
6.1 Volcano: Successful Plume	1.290 (273 K)	≈ 4.5 (1200 K)	$\approx 36.5^*$	125.0	b_0: Computed from MER $\approx 7.5 \times 10^5$ kg/s w_0, T_0, n_0: Thermo constraints [2, 3]
6.2 Volcano: Column Collapse	1.290 (273 K)	≈ 4.5 (1200 K)	$\approx 57.0^*$	40.0	Low momentum limit leading to collapse [3]

Table 1: Summary of the simulated scenarios and their initial boundary conditions. Part 1 validates the mathematical equations and numerical schemes (Boussinesq vs Non-Boussinesq). Part 2 uses real-world fluxes.

3 Results and Discussion

In this section, we present the numerical solutions obtained from our explicit Euler and RK4 solvers. First, we compare the Boussinesq and Non-Boussinesq approximations in theoretical conditions. Then we study real case scenarios: Deepwater Horizon Blowout (Gulf of Mexico, 2010) and Eyjafjallajökull Volcanic Eruption (Iceland, 2010).

3.1 Case 1: Simple Plume (Boussinesq Approximation)

In this first scenario, we simulate the classical Boussinesq plume (warm air injected into a standard atmosphere, as detailed in Table 1). To validate our numerical solver, we compare our numerical results with the analytical solution for a pure plume derived by Morton, Taylor, and Turner (1956) [1].

Since the pure plume analytical solution assumes an ideal point source ($b = 0$ at the origin), we introduce a virtual origin (z_v) to mathematically align it with our numerical simulation, which starts with a finite initial radius. Furthermore, the imposed initial velocity creates a "forced plume" (or buoyant jet) at the vent, rather than a pure plume.

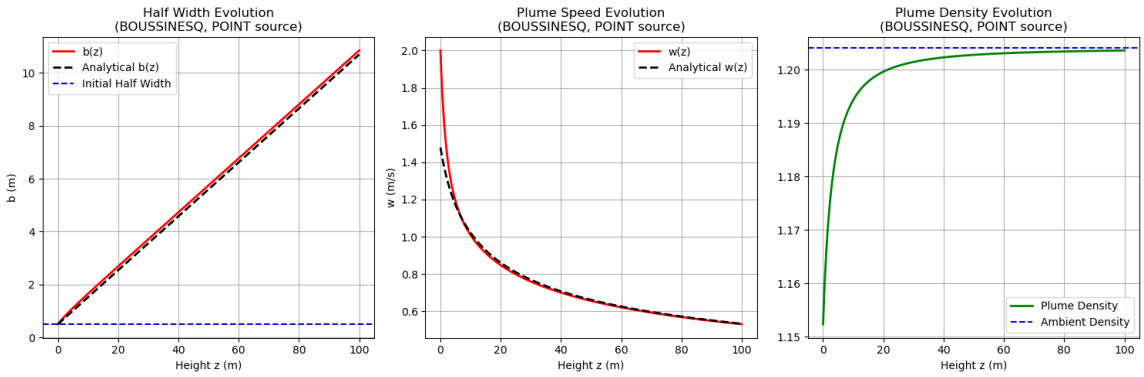

Figure 3: Spatial evolution of the vertical velocity $\langle w(z) \rangle$, plume radius $b(z)$, and mixture density $\rho(z)$ for Case 1. The numerical explicit Euler solutions (solid lines) are compared against the analytical pure plume solutions (dashed lines).

Figure 3 displays the spatial evolution of the macroscopic variables. The radius $b(z)$ exhibits the strictly linear expansion rate predicted by the theory, showing a perfect overlap between the numerical integration and the analytical model.

The velocity plot perfectly captures the dynamics of the forced plume. The vertical velocity $w(z)$ adjusts from its initial forced state at the source to merge with the behavior of the analytical pure plume. Because of the entrained air, the plume density tends asymptotically to the air density. This test successfully demonstrates the accuracy of our numerical scheme for Boussinesq cases.

3.2 Case 2: Light Plume (Non-Boussinesq)

In this scenario, we simulate a highly buoyant plume where the Boussinesq assumption ($\Delta\rho \ll \rho_a$) cannot be applied anymore. As detailed in Table 1, pure Helium ($\rho_0 = 0.18$ kg m^{-3}) is injected into standard ambient air. The Non-Boussinesq model uses the exact local density $\rho(z)$ for the plume's inertia and applies the density-corrected entrainment assumption.

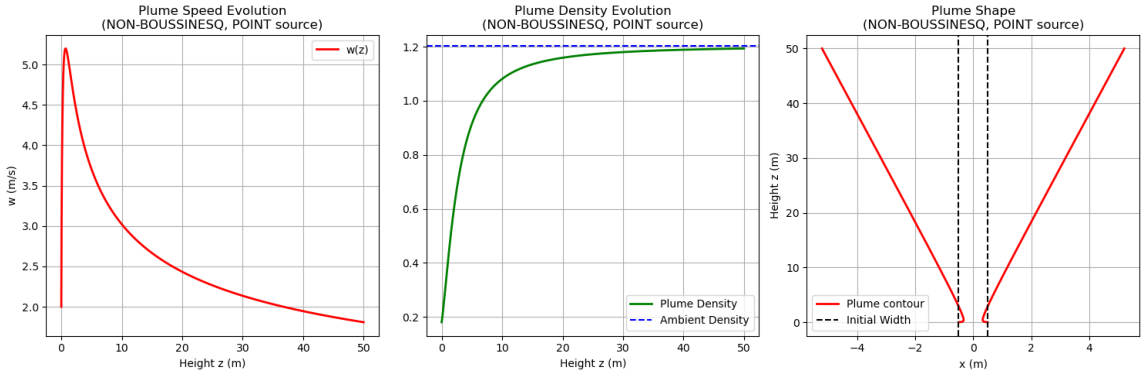

Figure 4: Spatial evolution of the vertical velocity $w(z)$, plume radius $b(z)$, and density $\rho(z)$ for the Non-Boussinesq plume (Helium).

Figure 4 shows the dynamics of a plume with an important initial density difference with ambient environment. Near the source, the massive buoyancy flux acts on a fluid with very low inertial mass ($\rho_0 \ll \rho_a$). Consequently, the vertical velocity $w(z)$ increases a lot at the beginning. A standard Boussinesq model, which artificially increases the plume's inertia by using ρ_a, would significantly underestimate this initial acceleration phase. Also, we can observe a "necking effect" on the plume shape, which is a specific behavior for such a plume.

As the plume rises, the modified entrainment mechanism rapidly incorporates the much denser ambient air. The density plot demonstrates how the mixture's density $\rho(z)$ increases from 0.18 kg m^{-3}. It asymptotically approaches the ambient atmospheric density ρ_a. Once sufficient air has been entrained and the density contrast becomes small enough, the velocity profile stabilizes. The plume radius $b(z)$ then resumes a steady linear expansion.

3.3 Numerical Implementation and Fountain Modeling

A fountain is defined as an upward jet of fluid with negative buoyancy ($\rho_0 > \rho_a$), causing the fluid to slow down under its own weight until it reaches a maximum stagnation height $z_{\max}$ before descending.

We simulated two distinct scenarios to test the limits of the Boussinesq approximation:

- **Case 4: A Heavy Gas Fountain (SF6) in Air**
- **Case 5: A Saline Water Fountain in Freshwater**

The solver termination condition is dictated by the flow physics: stagnation occurs when the vertical velocity vanishes. Numerically, we define $z_{\max}$ as the height where $w < 10^{-5}$ m/s.

3.3.1 Results Analysis & Graphics

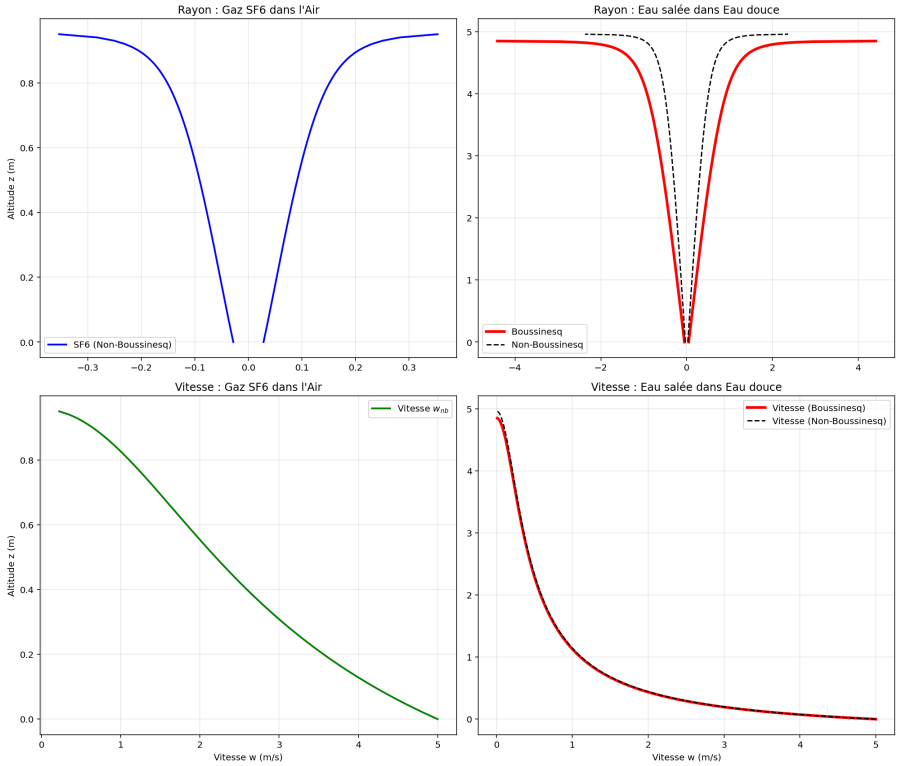

Figure 5: Vertical Velocity and Width Evolution of plumes in differents environnment

The analysis of the obtained graphs reveals two fundamental behaviors:

1. **Superposition in Water:** For the saline water case, the Boussinesq and Non-Boussinesq curves almost perfectly overlap. This validates the use of the classical approximation for small density contrasts.

2. **Rigidity of the SF6 Jet:** The heavy gas exhibits an almost constant radius over most of its ascent. Its inertia is such that mixing with the surrounding air is insufficient to significantly widen the jet cone until the very final stages of its trajectory.

3.3.2 Remarks and Model Limitations

- **The Downflow Phase:** An important limitation of our modeling lies in the absence of an explicit calculation of the descending phase. The physics of a real fountain is complex: it involves a two-way interaction between the rising core (*upflow*) and the surrounding descending fluid sheath (*downflow*), generating additional friction and entrainment [7].

  To maintain a tractable integral model, we restricted the analysis to the ascending phase. For the descending phase, we can anticipate that the velocity becomes negative and increases in magnitude due to gravity. We assume that the overall radius of the structure continues to grow by continued entrainment of the ambient fluid, in a manner that could be described as a "mirror symmetry" relative to the stagnation point.

- **Choice of Fluid Pairs (Miscibility):** The use of gas–gas systems (SF6/Air) or liquid–liquid systems (Saline/Freshwater) is deliberate. The MTT model equations rely on the fundamental *entrainment hypothesis*. It assumes that the jet fluid and ambient fluid are **miscible**: they mix intimately to form a continuous medium with gradually varying density. This fluid fusion is what allows the plume to "grow" as it rises.

- **The Limiting Case of Water in Air:** One might be tempted to model a water jet injected into air (extreme density ratio of 1000 vs. 1.2), but the MTT model is not suitable for this scenario.

  Injecting water into air produces a **two-phase flow**. Instead of seeing its density gradually change through mixing, the water jet retains its density of $1000\,\mathrm{kg/m^3}$ and eventually breaks into droplets. The equations in this project are intended for fluids that merge, such as smoke or oceanic currents.

3.4 Real-World Case Study: The Deepwater Horizon Disaster

We applied our stratified plume model to the *Deepwater Horizon* oil spill (Gulf of Mexico, 2010). This case is particularly interesting because, unlike fountains in a homogeneous environment, the oil does not necessarily rise directly to the surface: it can become trapped at depth due to the thermal and salinity stratification of the ocean.

3.4.1 Analysis of Results: The Trapping Phenomenon

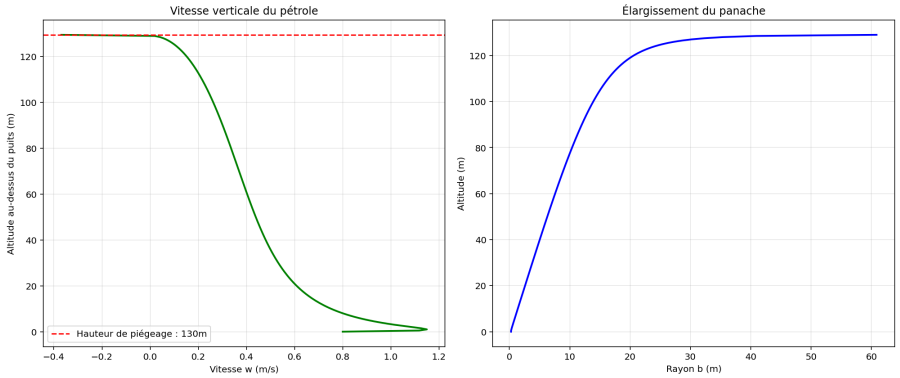

Figure 6: Vertical velocity & Width Evolution of oil plume in an oceanic environment

As shown in our graphs (Figure 6), the oil plume halts at approximately 130 m above the seabed (at a true depth of 1370 meters).

The vertical velocity exhibits a sharp drop. Unlike a plume in a homogeneous environment, the buoyancy flux F is not constant in this case. As the plume rises, it entrains dense deep seawater. Very quickly, the mixture density becomes equal to that of the surrounding water at higher levels: this defines the **neutral buoyancy point**.

The lateral spreading, as shown in the plot of the plume radius $b(z)$, indicates divergence. Physically, this means the oil can no longer rise. Since the volume flux continues to feed the plume from below, the oil is forced to spread horizontally.

3.5 Case 6: Thermodynamic Volcanic Plumes (Eyjafjallajökull)

We now apply our coupled thermodynamic model to the 2010 Eyjafjallajökull eruption scenario. Both subcases have the exact same measured Mass Eruption Rate ($Q_0 \approx 7.5 \times 10^5$ kg s^{-1}) and initial thermodynamic properties ($T_0 = 1200$ K, $n_0 = 0.04$). This results in a dense mixture initially much heavier than the atmosphere ($\rho_0 \approx 4.5$ kg m^{-3} $> \rho_a = 1.29$ kg m^{-3}). This way, flow dynamics depends only on the initial ejection speed.

3.5.1 Case 6.1: Successful Buoyant Plume (High Momentum)

In this scenario, the eruption is driven by a high exit velocity ($w_0 = 125$ m s^{-1}).

Figure 7 shows the behavior characteristic of explosive volcanic columns [3]. Initially, the mixture is negatively buoyant. However, the important initial momentum forces the heavy jet upwards. This creates a fast and massive entrainment of cold ambient air into the flow.

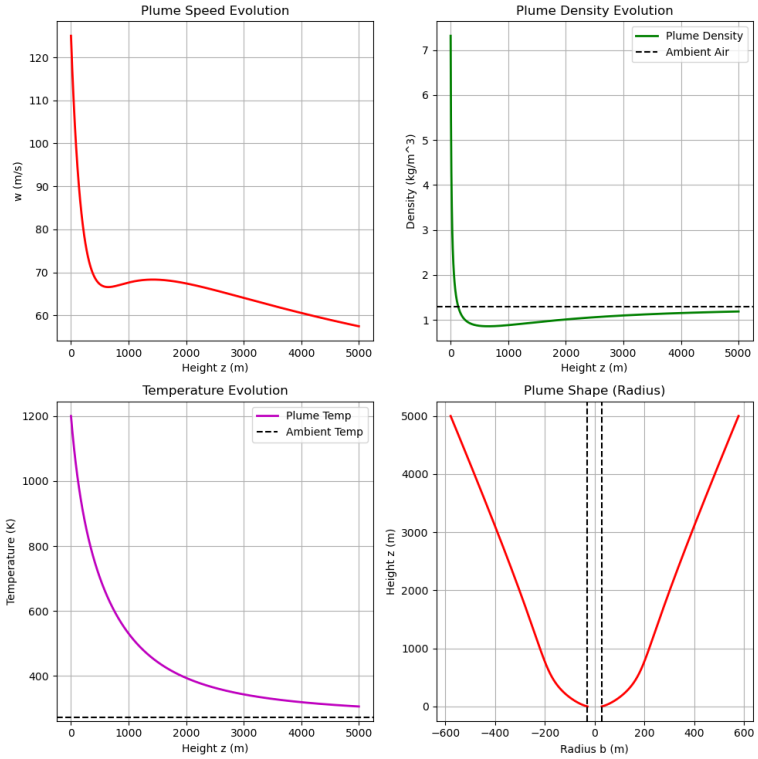

Figure 7: Spatial evolution of the successful volcanic plume (Case 6.1). The plots display the vertical velocity w, mixture density ρ, temperature T, and plume radius b as functions of the height z.

The entrained atmospheric air is instantly heated by the 1200 K solid ash particles. This heat transfer causes a violent thermal expansion of the gas phase. Consequently, it significantly reduces the density of the mixture. In the density plot we can see this thermodynamic expansion is powerful enough to drop the flow density below the ambient density ($\rho < 1.29$ kg m^{-3}). In this way, the flow becomes positively buoyant and it creates a plume instead of a fountain.

The continuous entrainment of ambient cold air reduces the plume temperature, making it tend to the ambient temperature : $T_a = 273$ K.

3.5.2 Case 6.2: Column Collapse (Low Momentum)

In contrast, we simulate an eruption phase where the exit velocity drops to $w_0 = 40$ m s^{-1}, which dynamically forces a wider volcano radius to satisfy the same mass flux.

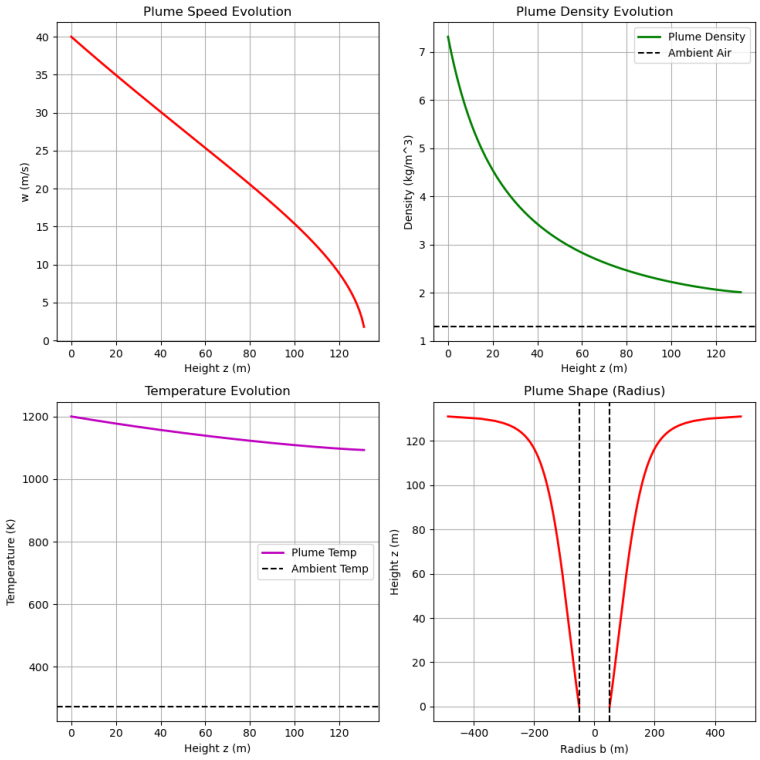

Figure 8: Detection of column collapse for a low-momentum eruption (Case 6.2). The vertical velocity $w(z)$ drops to zero before the density can fall below the ambient threshold.

As shown in Figure 8, the lower initial momentum changes the flow dynamics. While the mixture still entrains air and begins to heat it, the velocity drops to zero before the thermal expansion can fully invert the initial negative buoyancy.

The density plot shows that the mixture density remains higher than ρ_a. Consequently, the vertical velocity reaches $w = 0$ m s^{-1} at $z = 132$ m, triggering the numerical safeguard of our solver. This indicates an eruption column collapse, where the heavy mixture falls back to the ground.

4 Conclusion

In this mini-project, we had the opportunity to investigate entirely new topics. We first had to learn the fundamental dynamics of Boussinesq and non-Boussinesq plumes and fountains. For this, we reviewed the existing literature in these fields, from foundational papers, such as Morton, Taylor, and Turner (1956) [1], to more advanced studies, like Woods (1988, 2010) [2, 3]. We studied articles recommended by our professor as well as additional literature we found independently.

These theoretical foundations allowed us to identify and understand the conservation equations that describe turbulent plumes and fountains. We detailed these equations, demonstrating some of them to understand the physics as much as possible.

Then, we selected appropriate numerical integration methods: the Explicit Euler and Runge-Kutta (RK4) schemes. We implemented them in Python using the Spyder software. This allowed us to numerically solve the governing equations using standard entrainment assumptions and specific boundary conditions.

Once our Python solvers were finished, we defined several study scenarios. First, we analyzed theoretical cases to show the differences between Boussinesq and non-Boussinesq plumes and fountains. This showed the need to use the exact local density for flows with large initial density contrasts. Then, we applied our models to real-world scenarios, such as the 2010 Deepwater Horizon blowout and the Eyjafjallajökull volcanic eruption.

Through these simulations, we observed complex physical phenomena. Indeed, we have seen the trapping of oil plumes due to oceanic stratification. Another good result was the "buoyancy reversal" mechanism that we observed for volcanic ash columns.

However, due to time constraints, we had to make some simplifying assumptions. For instance, we restricted our study to steady-state conditions and uniform Top-Hat profiles, leaving aside time-dependent models and the more realistic Gaussian radial distributions. If we had more time, we would have explored new simulations that implement Gaussian profiles to better capture the turbulent radial diffusion of momentum and buoyancy. Furthermore, developing a transient solver would allow us to study the initial starting phase of a plume or its reaction to a changing eruption rate.

Finally, this project showed us the important role of fluid mechanics and numerical methods in understanding both industrial problems and natural geophysical events.

References

[1] Morton, B. R., Taylor, G. I., & Turner, J. S. (1956). Turbulent gravitational convection from maintained and instantaneous sources. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 234(1196), 1–24.

[2] Woods, A. W. (1988). The dynamics and thermodynamics of eruption columns. *Bulletin of Volcanology*, 50(3), 169–193.

[3] Woods, A. W. (2010). Turbulent plumes in nature. *Annual Review of Fluid Mechanics*, 42, 391–412.

[4] Socolofsky, S. A., Adams, E. E., & Sherwood, C. R. (2011). Formation dynamics of subsurface hydrocarbon intrusions following the Deepwater Horizon blowout. *Geophysical Research Letters*, 38(9).

[5] Crone, T. J., & Tolstoy, M. (2010). Magnitude of the 2010 Gulf of Mexico oil leak. *Science*, 330(6004), 634–634.

[6] Gudmundsson, M. T., Thordarson, T., et al. (2012). Ash generation and distribution from the April-May 2010 eruption of Eyjafjallajökull, Iceland. *Scientific Reports*, 2(1), 572.

[7] Bloomfield, L. J., & Kerr, R. C. (2000). A theoretical model of a turbulent fountain. *Journal of Fluid Mechanics*, 424, 1–22.

[8] List, E. J. (1982). Turbulent jets and plumes. *Annual Review of Fluid Mechanics*, 14(1), 189–212.

[9] Scase, M. M., Caulfield, C. P., Dalziel, S. B., & Hunt, J. C. R. (2006). Time-dependent plumes and jets with decreasing source strengths. *Journal of Fluid Mechanics*, 563, 443–461.

[10] van den Bremer, T. S., & Hunt, J. C. R. (2010). Universal solutions for Boussinesq and non-Boussinesq plumes. *Journal of Fluid Mechanics*, 644, 165–192.

[11] Woods, A. W. (1997). A model of vulcanian explosions. *Nature*, 389(6652), 685–687.

[12] Baines, W. D., & Turner, J. S. (1969). Turbulent buoyant convection from a source in a confined region. *Journal of Fluid Mechanics*, 37(1), 51–80.

[13] Rooney, G. G., & Linden, P. F. (1996). Similarity considerations for non-Boussinesq plumes in an unstratified environment. *Journal of Fluid Mechanics*, 318, 237–250.

A Appendices

A.1 Python Script : Deepwater Horizon Simulation

```
import numpy as np
import matplotlib.pyplot as plt

#================================================
# Partie 1 : Param tres Deepwater Horizon (R els)
#================================================
alpha = 0.1             # Coefficient d'entra nement (eau de mer)
g = 9.81
rho_pet = 850.0         # Densit  du p trole (kg/m3)
rho_ref = 1025.0        # Densit  de l'eau de mer    la source (1500m)
N2 = 5.0e-5               # Fr quence de Brunt-V is l   (stratification)

# Conditions    la source (le puits Macondo)
D_buse = 0.53            # Diam tre tube de forage
b0 = D_buse / 2.0        # Rayon tube de forage
w0 = 0.8                 # Vitesse de sortie estim e (m/s)

# Flux initiaux (Upflow)
Q0 = np.pi * (b0**2.0) * w0
M0 = np.pi * (b0**2.0) * (w0**2.0)
g_prime0 = g * (rho_ref - rho_pet) / rho_ref
F0 = Q0 * g_prime0

#================================================
# Partie 2 : Syst me Diff rentiel Boussinesq
#================================================
def deriveur(Q, M, F, a, N2):
    if M <= 1.0e-10: return 0, 0, 0
    dQdz = 2.0 * alpha * np.sqrt(np.pi * M)
    dMdz = (F * Q) / M
    dFdz = - Q * N2
    return dQdz, dMdz, dFdz

#================================================
# Partie 3 : R solution par boucle RK4
#================================================
def rk4(Q_init, M_init, F_init, N2):
    dz = 0.5            # Pas de 50cm
    z = 0.0
    z_list, Q_list, M_list, F_list = [0], [Q_init], [M_init], [F_init]
    Q, M, F = Q_init, M_init, F_init

    while M > 1.0e-6 and z < 1500.0:
        k1 = deriveur(Q, M, F, alpha, N2)
        k2 = deriveur(Q + dz/2*k1[0], M + dz/2*k1[1], F + dz/2*k1[2], alpha, N2)
        k3 = deriveur(Q + dz/2*k2[0], M + dz/2*k2[1], F + dz/2*k2[2], alpha, N2)
        k4 = deriveur(Q + dz*k3[0], M + dz*k3[1], F + dz*k3[2], alpha, N2)

        Q += (dz/6) * (k1[0] + 2*k2[0] + 2*k3[0] + k4[0])
        M += (dz/6) * (k1[1] + 2*k2[1] + 2*k3[1] + k4[1])
        F += (dz/6) * (k1[2] + 2*k2[2] + 2*k3[2] + k4[2])
        z += dz

        z_list.append(z)
        Q_list.append(Q)
        M_list.append(M)
```

```
        F_list.append(F)

        if F < 0.0 and M < 0.01:
            break

    return np.array(z_list), np.array(Q_list), np.array(M_list), np.array(F_list)

# Ex cution et Post-traitement
z_vec, Q_vec, M_vec, F_vec = rk4(Q0, M0, F0, N2)
w_vec = M_vec / Q_vec
b_vec = Q_vec / np.sqrt(np.pi * M_vec)
z_max = z_vec[-1]

# Affichage des r sultats
fig, axs = plt.subplots(1, 2, figsize=(14, 6))
axs[0].plot(w_vec, z_vec, 'g', lw=2)
axs[0].axhline(z_max, color='r', linestyle='--')
axs[1].plot(b_vec, z_vec, 'b', lw=2)
plt.show()
```

A.2 Python Script : Comparison between SF6 Gas and Saline Water

```
import numpy as np
import matplotlib.pyplot as plt

#================================================
# Partie 1 : PARAMETRES ET SOLVEUR (RK4)
#================================================
alpha = 0.085
g = 9.81
dz = 0.01
z_max = 5.0

def rk4(deriv_fonc, z0, y0, dz, z_max):
    z_vals, y_vals = [z0], [y0]
    z, y = z0, np.array(y0)

    while z < z_max:
        k1 = np.array(deriv_fonc(z, y))
        k2 = np.array(deriv_fonc(z + dz/2, y + (dz/2) * k1))
        k3 = np.array(deriv_fonc(z + dz/2, y + (dz/2) * k2))
        k4 = np.array(deriv_fonc(z + dz, y + dz * k3))

        y_next = y + (dz/6) * (k1 + 2*k2 + 2*k3 + k4)
        # Arret automatique au sommet de la fontaine (vitesse nulle)
        if y_next[1] < 1e-5: break

        y, z = y_next, z + dz
        z_vals.append(z)
        y_vals.append(y)
    return np.array(z_vals), np.array(y_vals).T

#=====================================================
# Partie 2 : CAS NON-BOUSSINESQ : GAZ SF6 DANS L'AIR
#=====================================================
rho_air = 1.225
rho_sf6 = 6.12
b0 = 0.05
w0 = 5.0
```

```
38
39 Q0_nb = rho_sf6 * (b0**2) * w0
40 M0_nb = rho_sf6 * (b0**2) * (w0**2)
41 F0_nb = g * (b0**2) * w0 * (rho_air - rho_sf6)
42
43 def deriveur(z, y):
44     Q, M, F = y
45     dQdz = 2 * alpha * np.sqrt(rho_air * M)
46     dMdz = (F * Q) / M
47     return [dQdz, dMdz, 0]
48
49 z_nb, y_nb = rk4(deriveur, 0, [Q0_nb, M0_nb, F0_nb], dz, z_max)
50
51 # Conversion des flux en grandeurs physiques
52 v_flux_nb = (y_nb[2]/g + y_nb[0]) / rho_air
53 w_nb = y_nb[1] / y_nb[0]
54 b_nb = np.sqrt(v_flux_nb / (np.pi * w_nb))
55
56 #============================================================
57 # Partie 3.1 : CAS BOUSSINESQ : Eau salee dans eau douce
58 #============================================================
59 rho_douce = 1000.0
60 rho_salee = 1030.0
61 g_p = g * (rho_douce - rho_salee) / rho_douce
62
63 Q0_b = np.pi * (b0**2) * w0
64 M0_b = np.pi * (b0**2) * (w0**2)
65 F0_b = Q0_b * g_p
66
67 def deriveur2(z, y):
68     Q, M, F = y
69     dQdz = 2 * alpha * np.sqrt(np.pi * M)
70     dMdz = (F * Q) / M
71     return [dQdz, dMdz, 0]
72
73 z_b, y_b = rk4(deriveur2, 0, [Q0_b, M0_b, F0_b], dz, z_max)
74 w_b = y_b[1] / y_b[0]
75 b_b = np.sqrt(y_b[0] / (np.pi * w_b))
76
77 #============================================================
78 # Partie 3.2 : CAS NON-BOUSSINESQ : Eau salee dans eau douce
79 #============================================================
80 Q0_eau_nb = rho_salee * (b0**2) * w0
81 M0_eau_nb = rho_salee * (b0**2) * (w0**2)
82 F0_eau_nb = g * (b0**2) * w0 * (rho_douce - rho_salee)
83
84 def deriveur3(z, y):
85     Q, M, F = y
86     dQdz = 2 * alpha * np.sqrt(rho_douce * M)
87     dMdz = (F * Q) / M
88     return [dQdz, dMdz, 0]
89
90 z_wnb, y_wnb = rk4(deriveur3, 0, [Q0_eau_nb, M0_eau_nb, F0_eau_nb], dz, z_max)
91
92 v_flux_wnb = (y_wnb[2]/g + y_wnb[0]) / rho_douce
93 w_wnb = y_wnb[1] / y_wnb[0]
94 b_wnb = np.sqrt(v_flux_wnb / (np.pi * w_wnb))
```

A.3 Python Script : Boussinesq and Non-Boussinesq Script for Point and Line Sources

```
    import numpy as np
import matplotlib.pyplot as plt

# This code is meant to be able to perform calculation for various cases
# editing Model configuration section and boundary conditions

# Model configuration
GEOMETRY = "POINT"       # Defining source geometry : "POINT" or "LINE"
APPROXIMATION = "NON-BOUSSINESQ" # Defining if we have boussinesq approx. : "
    BOUSSINESQ" or not : "NON-BOUSSINESQ"

#Defining constants
g = 9.81
alpha = 0.085       # Entrainment coefficient
rho_a = 1.204       # Ambient density

#Space discretization
hmax = 50 # Maximum height
N = 3000 # Number of points
dz = hmax/N
z = np.linspace(0, hmax, N)

#Defining boundary conditions
rho0 = 0.18         # Plume density at source
b0 = 0.5            # Source radius or half-width (m)
w0 = 2.0            # Plume speed at source

#Defining rho value to calculate Q0, M0 and F0.
#If BOUSSINESQ, we use ambiant density, else we use plume density
if (APPROXIMATION == "BOUSSINESQ"):
    rho_init = rho_a
elif (APPROXIMATION == "NON-BOUSSINESQ"):
    rho_init = rho0

# Calculculating Q0, M0 and F0 depending if it is a Point source or a line source
if GEOMETRY == "POINT":
    # Q = rho * b^2 * w
    Q0 = rho_init * (b0**2) * w0
    M0 = rho_init * (b0**2) * (w0**2)
    F0 = (rho_a - rho0) * (b0**2) * w0 * g
elif GEOMETRY == "LINE":
    # Q = rho * b * w
    Q0 = rho_init * b0 * w0
    M0 = rho_init * b0 * (w0**2)
    F0 = (rho_a - rho0) * b0 * w0 * g

y0 = [Q0, M0, F0]
R = np.zeros((3,N))
R[:,0] = y0

collapse_index = N-1

# Defining explicit euler method to solve the equations
def euler(Y):
    Qn, Mn, Fn = Y

    wn = Mn / Qn # Defining wn to have a more compact code
```

```
    # Calculating eta = rho/rho_a (as defined in references)
    if APPROXIMATION == "BOUSSINESQ":
        eta = 1
    else:
        eta = 1 / (1 + Fn / (g * Qn)) # =rho/rho_a

    if GEOMETRY == "POINT":
        Qn1 = Qn + dz * 2 * alpha * np.sqrt(Mn * rho_a)
    elif GEOMETRY == "LINE":
        Qn1 = Qn + dz * 2 * alpha * rho_a * wn * np.sqrt(eta)

    Mn1 = Mn + dz * Fn * Qn / Mn

    return np.array([Qn1, Mn1, Fn])

# Calculating Q, M, F using explicit euler method
for i in range (N-1):
    if R[1, i] <= 0:
        collapse_index = i
        print(f"COLUMN COLLAPSE DETECTED at z = {z[i]:.1f} meters")
        break

    R[:, i+1] = euler(R[:, i])

# Extracting results and truncating results if fountain
z = z[:collapse_index]
Q = R[0, :collapse_index]
M = R[1, :collapse_index]
F = R[2, :collapse_index]

# Calculating b, w and rho
w = M / Q
rho = rho_a / (1.0 + F / (g * Q))

if GEOMETRY == "POINT":

    if (APPROXIMATION == "BOUSSINESQ"):
        b = np.sqrt(Q / (rho_a * w))
    else:
        b = np.sqrt(Q / (rho * w))

elif GEOMETRY == "LINE":
    if (APPROXIMATION == "BOUSSINESQ"):
        b = Q / (rho_a * w)
    else:
        b = Q / (rho * w)

# Calculating Analyctical Solution if Boussinesq
if GEOMETRY == "POINT" and APPROXIMATION == "BOUSSINESQ":
    #Calculating Virtual Origin (zv) to match initial radius b0
    zv = (5 * b0) / (6 * alpha)
    z_virt = z + zv

    #Specific buoyancy flux
    f0 = F0 / rho_a

    #Solutions for Pure Plume
    b_ana = (6 * alpha / 5) * z_virt
    w_ana = (5 / (6 * alpha)) * ((9 * alpha * f0 / 10)**(1/3)) * (z_virt**(-1/3))
```

```
117
118     q_ana = (6 * alpha / 5) * ((9 * alpha * f0 / 10)**(1/3)) * (z_virt**(5/3))
119     Q_ana = rho_a * q_ana
120     rho_ana = rho_a - (F0 * rho_a) / (g * Q_ana)
121
122
123 plotColumns = 3
124 plotLines = 1
125 nPlots = 0
126 #Plotting results
127 plt.figure(figsize=(plotColumns*5, plotLines*5))
128
129 # # Plume width
130 # nPlots+=1
131 # plt.subplot(plotLines, plotColumns, nPlots)
132 # plt.plot(z, b, 'r-', linewidth=2, label='b(z)')
133 # if GEOMETRY == "POINT" and APPROXIMATION == "BOUSSINESQ":
134 #     plt.plot(z, b_ana, 'k--', linewidth=2, label='Analytical b(z)')
135 # plt.axhline(b0, color='b', linestyle='--', label='Initial Half Width')
136 # plt.title(f"Half Width Evolution\n({APPROXIMATION}, {GEOMETRY} source)")
137 # plt.xlabel("Height z (m)")
138 # plt.ylabel("b (m)")
139 # plt.grid(True)
140 # plt.legend()
141
142 # Plume speed
143 nPlots+=1
144 plt.subplot(plotLines, plotColumns, nPlots)
145 plt.plot(z, w, 'r-', linewidth=2, label='w(z)')
146 if GEOMETRY == "POINT" and APPROXIMATION == "BOUSSINESQ":
147     plt.plot(z, w_ana, 'k--', linewidth=2, label='Analytical w(z)')
148 plt.title(f"Plume Speed Evolution\n({APPROXIMATION}, {GEOMETRY} source)")
149 plt.xlabel("Height z (m)")
150 plt.ylabel("w (m/s)")
151 plt.grid(True)
152 plt.legend()
153
154 # Plume density
155 nPlots+=1
156 plt.subplot(plotLines, plotColumns, nPlots)
157 plt.plot(z, rho, 'g-', linewidth=2, label='Plume Density ')
158 plt.axhline(rho_a, color='b', linestyle='--', label='Ambient Density')
159 plt.title(f"Plume Density Evolution\n({APPROXIMATION}, {GEOMETRY} source)")
160 plt.xlabel("Height z (m)")
161 plt.grid(True)
162 plt.legend()
163
164 # Plume shape
165 nPlots+=1
166 plt.subplot(plotLines, plotColumns, nPlots)
167 plt.plot(b, z, 'r-', linewidth=2, label = 'Plume contour')
168 plt.plot(-b, z, 'r-', linewidth=2)
169 plt.axvline(b0, color='k', linestyle='--', label='Initial Width')
170 plt.axvline(-b0, color='k', linestyle='--')
171 plt.title(f"Plume Shape\n({APPROXIMATION}, {GEOMETRY} source)")
172 plt.xlabel("x (m)")
173 plt.ylabel("Height z (m)")
174 plt.grid(True)
175 plt.legend()
176
```

```
# # LogLog plot
# nPlots=+1
# plt.subplot(plotLines, plotColumns, nPlots)
# plt.loglog(z[1:], b[1:], 'r-', linewidth=2, label='b(z)')

# # Checking for 1/6 slope for point source and non-boussinesq case
# if (GEOMETRY == "POINT") and (APPROXIMATION == "NON-BOUSSINESQ"):
#     plt.loglog(z[1:], z[1:]**(1/6), 'k--', label='1/6 Slope')

# plt.loglog(z[1:], z[1:], 'y--', label='1 Slope (Linear)')

# plt.title(f"Evolution (Log-Log)\n({APPROXIMATION}, {GEOMETRY} source)")
# plt.xlabel("Height z (m)")
# plt.grid(True, which="both")
# plt.legend()

plt.tight_layout()
plt.show()
```

A.4 Python Script : Volcanic Case

```
    import numpy as np
import matplotlib.pyplot as plt

#Defining constants
g = 9.81
alpha = 0.085       # Entrainment coefficient
rho_a = 1.29        # Ambient density
T_a = 273.0         # Ambient temperature

# Defining Ashes and Magma properties
T_0 = 1200.0        # Initial eruption temperature
n0 = 0.04           # Initial gas mass fraction (4% gas, 96% solid ash)
rho_s = 2500.0      # Density of solid ash particles
Cp_a = 1000.0       # Specific heat capacity of air/gas
Cp_s = 1100.0       # Specific heat capacity of solid ash

# Boundary conditions (From Litterature)
Q0 = 7.5e5          # Measured MER
w0 = 125.0           # Eruption exit speed

# Calculating initial mixture properties
Cp_0 = n0 * Cp_a + (1 - n0) * Cp_s
rho_g0 = rho_a * (T_a / T_0)
rho0 = 1.0 / (n0 / rho_g0 + (1 - n0) / rho_s)

# Calculating b0 in function of the other parameters
b0 = np.sqrt(Q0 / (rho0 * w0))

# Calculating initial fluxes
M0 = rho0 * (b0**2) * (w0**2)  # Ce qui revient exactement    M0 = Q0 * w0
H0 = Q0 * Cp_0 * T_0
Flux_Particules = Q0 * (1 - n0)

#Space discretization
hmax = 5000 # Maximum height
N = 5000
dz = hmax/N
z = np.linspace(0, hmax, N)
```

```

y0 = [Q0, M0, H0]
R = np.zeros((3, N))
R[:, 0] = y0
collapse_index = N - 1

# Defining explicit Euler method
def euler(Y):
    Qn, Mn, Hn = Y

    # Local variables calculation
    bn = np.sqrt(Qn**2 / (rho0 * Mn))

    # Local mass fractions and specific heat
    masse_gaz = Qn - Flux_Particules
    n = masse_gaz / Qn
    Cp_mix = n * Cp_a + (1 - n) * Cp_s

    # Local Temperature and Density
    Tn = Hn / (Qn * Cp_mix)
    rho_g = rho_a * (T_a / Tn)
    rho_n = 1.0 / (n / rho_g + (1 - n) / rho_s)

    # Exact local radius
    bn = np.sqrt(Qn**2 / (rho_n * Mn))

    # Differential equations (Woods 2010)
    dQ = dz * 2 * alpha * np.sqrt(Mn * rho_a)
    dM = dz * g * (rho_a - rho_n) * bn**2
    dH = dQ * Cp_a * T_a  # Entrained air brings its ambient heat

    return np.array([Qn + dQ, Mn + dM, Hn + dH])

# Solving system using explicit Euler method
for i in range (N-1):
    # Collapse detection : if M<0, collapsing => need to stop calculation
    if R[1, i] <= 0:
        collapse_index = i
        print(f"Collapse detected at z = {z[i]:.1f} meters")
        break

    R[:, i+1] = euler(R[:, i])


# Extracting and truncating results (if collapse) (because R was initialized as np.
    zeros)
z = z[:collapse_index]
Q = R[0, :collapse_index]
M = R[1, :collapse_index]
H = R[2, :collapse_index]

# Recalculating physical variables for plotting
w = M / Q
n_array = (Q - Flux_Particules) / Q
Cp_array = n_array * Cp_a + (1 - n_array) * Cp_s
T_array = H / (Q * Cp_array)

rho_g_array = rho_a * (T_a / T_array)
rho = 1.0 / (n_array / rho_g_array + (1 - n_array) / rho_s)
b = np.sqrt(Q**2 / (rho * M))
```

```
98
99  #Plotting results
100 plt.figure(figsize=(10, 10))
101
102 # Plume Speed
103 plt.subplot(2, 2, 1)
104 plt.plot(z, w, 'r-', linewidth=2)
105 plt.title("Plume Speed Evolution")
106 plt.xlabel("Height z (m)")
107 plt.ylabel("w (m/s)")
108 plt.grid(True)
109
110 # Plume Density
111 plt.subplot(2, 2, 2)
112 plt.plot(z, rho, 'g-', linewidth=2, label='Plume Density')
113 plt.axhline(rho_a, color='k', linestyle='--', label='Ambient Air')
114 plt.title("Plume Density Evolution")
115 plt.xlabel("Height z (m)")
116 plt.ylabel("Density (kg/m^3)")
117 plt.legend()
118 plt.grid(True)
119
120 # Temperature
121 plt.subplot(2, 2, 3)
122 plt.plot(z, T_array, 'm-', linewidth=2, label='Plume Temp')
123 plt.axhline(T_a, color='k', linestyle='--', label='Ambient Temp')
124 plt.title("Temperature Evolution")
125 plt.xlabel("Height z (m)")
126 plt.ylabel("Temperature (K)")
127 plt.legend()
128 plt.grid(True)
129
130 # Plume Shape
131 plt.subplot(2, 2, 4)
132 plt.plot(b, z, 'r-', linewidth=2)
133 plt.plot(-b, z, 'r-', linewidth=2)
134 plt.axvline(b0, color='k', linestyle='--')
135 plt.axvline(-b0, color='k', linestyle='--')
136 plt.title("Plume Shape (Radius)")
137 plt.xlabel("Radius b (m)")
138 plt.ylabel("Height z (m)")
139 plt.grid(True)
140
141 # # Gas Mass Fraction
142 # plt.subplot(2, 3, 5)
143 # plt.plot(z, n_array * 100, 'c-', linewidth=2) # Converted to %
144 # plt.axhline(n0 * 100, color='k', linestyle='--', label='Initial Gas %')
145 # plt.title("Gas Mass Fraction")
146 # plt.xlabel("Height z (m)")
147 # plt.ylabel("Gas %")
148 # plt.grid(True)
149 # plt.legend()
150
151 plt.tight_layout()
152 plt.show()
```